

Informe P5 - Grupo 8

1. Objetivo y filosofía: Entender

El objetivo de esta práctica era introducirnos en la "caja negra" de los Modelos de Lenguaje para ver cómo funcionan por dentro desde cero. Nuestro objetivo no era entrenar un modelo gigantesco que compitiera con los sistemas actuales, sino entender íntimamente, línea a línea, qué piezas matemáticas y de software hacen falta para construir un pipeline de Procesamiento del Lenguaje Natural funcional.

Para ello, hemos construido dos sistemas interconectados.

- Primero, un **modelo de lenguaje causal** capaz de generar texto de forma autorregresiva.
- Segundo, un modelo de **Reconocimiento de Entidades Nombradas (NER)** que reutiliza el conocimiento sintáctico aprendido en la primera fase. Todo el sistema ha sido programado desde cero, entrenado en CPU y encapsulado en una herramienta de línea de comandos completa y fácil de usar.

En este informe, detallamos nuestras decisiones de diseño, los experimentos realizados, y nuestras reflexiones sobre los obstáculos que fuimos superando.

2. El primer paso: Enseñar a leer (Tokenización BPE)

Un modelo no puede procesar lenguaje si antes no decidimos cómo trocearlo. Una opción es tokenizar por palabras completas, pero que descartamos porque se generaría entonces un vocabulario inmanejable y, además, el modelo no sabría qué hacer con palabras nuevas que entraran. Otra opción puede ser tokenizar por caracteres individuales, pero entonces las secuencias serían tan largas que el modelo perdería el hilo del significado.

Como en el punto medio está la virtud, nuestra solución fue implementar nuestro propio tokenizador BPE (*Byte Pair Encoding*). Lo entrenamos iterativamente para fusionar los pares de caracteres más frecuentes de los textos de Lewis Carroll (estamos refiriéndonos a: *Alicia en el País de las Maravillas* y *A Través del Espejo*).

Ejemplo de nuestro BPE: Si pasamos la frase "*Alice went to Wonderland*" por nuestro analizador:

- Caracteres originales: 24
- Tokens resultantes: 11
- Ratio de comprensión: 2.18 caracteres por token

Para comprobarlo empíricamente, podemos introducir esta instrucción en nuestra línea de comandos:

```
uv run fdi-pln-2608-p5 analyze-bpe --weights checkpoints/p5_causal_2608.pth \
--text "Alice went to Wonderland"
```

Se pueden probar otras frases sustituyéndolas en el apartado de "text".

Al inspeccionar los tokens, notamos que algunas palabras con mayúsculas se segmentan de forma poco natural. Aquí hemos encontrado una limitación que, sin embargo, era esperable al entrenar un BPE con un corpus tan reducido. Pero matemáticamente cumple su propósito a la perfección: mantener un vocabulario pequeño y poder codificar cualquier texto de entrada.

3. Construyendo el cerebro: La Arquitectura Transformer

Fuimos implementando la arquitectura bloque a bloque: embeddings posicionales, atención multi-cabezal y capas feed-forward con GELU. Pero si tuviéramos que destacar el aprendizaje más importante de esta fase, fue comprender la necesidad de la **atención escalada**.

La atención es el verdadero motor del Transformer, ya que es el mecanismo que permite al modelo determinar qué palabras de una secuencia son más importantes para cada una de las demás. Para lograrlo, cada posición del texto genera tres representaciones: *Queries* (Q), *Keys* (K) y *Values* (V). Con ellas se calcula la afinidad entre tokens mediante la Atención de Producto Escalar Escalado (*Scaled Dot-Product Attention*):

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Al programar esta ecuación nos dimos cuenta de que la teoría tiene una traducción técnica vital. Entendimos el verdadero valor de dividir por la raíz cuadrada de la dimensión (`sqrt(head_dim)` o $\sqrt{d_k}$). Sin este escalado matemático, los productos escalares entre nuestras *Queries* y *Keys* crecían desproporcionadamente; esto saturaba la función Softmax, empujaba los gradientes a cero y paralizaba por completo el aprendizaje de la red en nuestra CPU.

Utilizamos este mismo "cerebro" de dos formas distintas, cambiando solo el flujo de la información:

- **Para generar texto (Causal):** Aplicamos una máscara para que el modelo solo pudiera "mirar hacia atrás" (matemáticamente, esto es una matriz triangular inferior). Para predecir la siguiente palabra, no puedes hacer trampas mirando el futuro.
- **Para buscar entidades (NER):** Eliminamos la máscara. Para clasificar si una palabra es un personaje o un lugar, la red necesita un contexto **bidireccional** (leer tanto el inicio como el final de la oración).

4. Fase 1: Entrenamiento Causal y Experimentación

Diseñamos una configuración deliberadamente pequeña para que los tiempos de entrenamiento en CPU fueran asumibles, aceptando el sacrificio en la capacidad del modelo para memorizar textos largos.

Nuestra configuración final (p5_causal_2608.pth):

- `context_size = 128` | `seq_len = 128` | `vocab_size = 300`
- `d_model = 128` | `n_heads = 4` | `n_layers = 4`
- `batch_size = 64` | `epochs = 5` | `lr = 0.0003` | `dropout = 0.1`

El impacto de la Temperatura y el Top-k

Entrenar el modelo es solo la mitad del trabajo; la otra mitad es saber cómo extraerle el texto. Realizamos varias pruebas cambiando los parámetros de muestreo a partir del prompt "`Alice was`", limitando la salida a 40 tokens.

Tanto la temperatura como el top-k son mecanismos que intervienen en el último paso del modelo: el muestreo (sampling).

Cuando tu Transformer procesa un texto, no predice una única palabra ganadora. Lo que hace es generar un vector enorme (los logits) y aplicarles una función Softmax para asignar una probabilidad a cada palabra del vocabulario. Estos dos parámetros alteran esa distribución de probabilidades antes de que "tiremos el dado" para elegir la siguiente palabra. Normalmente se usan a la vez porque hacen un equipo perfecto:

- El Top-k actúa primero como filtro: expulsa a las palabras completamente absurdas para asegurar que la gramática no se rompa.
- La Temperatura actúa después sobre las candidatas "buenas" que han sobrevivido al filtro: decide si tiramos los dados de forma arriesgada (alta) o conservadora (baja) entre ese grupo de finalistas.

Temp	Top-k	Texto generado (extracto literal)	Nuestra lectura analítica
0.5	10	"Alice was a long way to find that they must have to do that!" "i haven't tell you see, oh, if"	Demasiado conservador. La red elige los caminos más seguros. El resultado es gramaticalmente estable pero repetitivo y poco imaginativo.
0.8	10	"Alice was a pawn out of it, and she could not thought to herself, "if it would have a good must grine,"	El punto dulce. Permite cierta exploración del vocabulario aportando variedad y "estilo Lewis Carroll", sin destruir por completo la estructura de la oración.
1.2	20	"*Alice was all thank croquet, which the time with so. * * * *"	Colapso del modelo. Forzar a una red tan pequeña a explorar tokens poco probables (alta temperatura) añade demasiado ruido; pierde el hilo y entra en bucles de puntuación.

Conclusión: Un modelo pequeño no tiene representaciones lingüísticas robustas. Subir la temperatura rompe la coherencia rapidísimo. Por eso, fijamos los valores por defecto de nuestra aplicación en `temperature=0.8` y `top-k=20`.

5. Fase 2: Reciclando conocimiento (Transfer Learning para NER)

En lugar de entrenar un clasificador NER desde cero, aplicamos Transfer Learning. Cargamos los pesos de nuestro modelo causal (que ya sabe cómo se estructuran las palabras en inglés) y le acoplamos una cabeza lineal nueva para predecir etiquetas BIO (`PER` y `LOC`).

Utilizamos los datos que anotamos a mano en la preentrega junto al Grupo 1, donde aplicamos una clasificación BIO adaptada para distinguir inicios y continuaciones de personajes (`pi`, `pc`) y lugares (`li`, `lc`), convirtiéndolos finalmente de forma automática al estándar CoNLL.

Resultados de la evaluación (`p5_ner_2608.pth`):

Métrica	Token-level	Entity-level
Accuracy	0.8881	-
Precision	0.2666	0.1132
Recall	0.7053	0.5909
F1-Score	0.3869	0.1901

Desglose por clase:

Tipo	Precision	Recall	F1	Soporte (Ejemplos reales)
LOC	0.1027	0.8824	0.1840	17
PER	0.1168	0.5376	0.1919	93

Análisis de las métricas

La altísima *accuracy* (89%) es un espejismo estadístico provocado por el fuerte desbalance de clases (casi todo el texto es la clase "O"). Las métricas a nivel de entidad cuentan la historia real.

Sin embargo, tener un *Recall* de casi el 60% es un éxito: demuestra que el modelo **sí aprende a localizar entidades**, encontrando con facilidad personajes como *Alice* o el *Hatter*. El problema radica en la precisión (~11%). Al introducir pesos de clase en el entrenamiento para evitar que el modelo ignorase a las minorías, creamos un modelo excesivamente "valiente" que marca demasiados falsos positivos (como preposiciones o palabras de enlace).

La decisión del Post-procesado

Nos dimos cuenta de que, aunque el modelo encontraba las entidades correctas, la salida en bruto de la inferencia era muy ruidosa y frustrante para un usuario.

Para solucionar esto, añadimos una pequeña capa heurística de post-procesado **solo en el momento de la inferencia final** (no afecta a los pesos ni altera las métricas de evaluación mostradas arriba). Este post-procesado:

1. Descarta palabras funcionales evidentes (*the, ran, through*).
2. Normaliza convenciones del dominio (asegurando que *Queen* o *White Rabbit* se marquen como **PER**, y *Wonderland* como **LOC**).
3. Agrupa palabras consecutivas en una sola entidad visual (por ejemplo, si la red escupía [**White**, **PER**] y [**Rabbit**, **PER**], lo unifica como [**White Rabbit**, **PER**]).

Gracias a esto, convertimos una predicción ruidosa en una herramienta por terminal (**CLI**) verdaderamente útil y clara, demostrando que en el PLN real, el Deep Learning y las reglas heurísticas a menudo trabajan de la mano.

6. Ingeniería de Software y Reproducibilidad

Todo este pipeline no tendría valor si solo funcionara en nuestros ordenadores. Invertimos tiempo en asegurar que la práctica fuera un producto completo:

- **Checkpoints autocontenidos:** Guardamos los pesos, la configuración y el tokenizador BPE en un mismo archivo `.pth`.
 - **Interfaz CLI:** Construimos un menú interactivo con `Typer` y `Rich` que permite generar texto, evaluar o inferir sin tener que memorizar comandos interminables.
 - **Empaquetado:** Todo el proyecto compila de forma limpia en un `.whl` a través de `uv build`.
-

7. Conclusiones Finales

Las limitaciones que hemos documentado (deformaciones léxicas, pérdida de coherencia a largo plazo, o falsos positivos en NER) no son fracasos; son la consecuencia matemática esperable de entrenar una arquitectura reducida, con un corpus literario pequeño, en una CPU local.

Sin embargo, el objetivo didáctico se ha cumplido con creces. Hemos comprobado "manualmente" cómo las representaciones aprendidas por un BPE viajan por los bloques de atención, cómo la temperatura modifica la creatividad de la red, y cómo un modelo generativo puede ser reciclado para extraer información estructurada (NER). La Inteligencia Artificial actual se asienta sobre estas mismas piezas; nosotras, simplemente, hemos construido nuestra propia maqueta a escala.

Integrantes: Marina Triviño de las Heras | Carlota Salazar Martín (Grupo 2608)